LAPORAN PRAKTIKUM

STRUKTUR DATA

“Single Linked List”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul "Single Linked List"

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 07 Mei 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat 2

BAB II PEMBAHASAN 3

1.1 Pengertian Single Linked List 3

2.2 Implementasi Pada Java 3

2.2.1 NodeSLL_2511533020 3

2.2.2 SLL_2511533020 4

2.2.3 PencarianSLL_2511533020 8

2.2.4 HapusSLL_2511533020 7

BAB III PENUTUP 6

3.1 Kesimpulan 6

3.2 Saran 6

3.3 Penutup 6

DAFTAR PUSTAKA 11

TUGAS PRAKTIKUM 12

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam mata kuliah Struktur Data, pemahaman tentang cara menyimpan data secara dinamis merupakan hal yang fundamental. Salah satu struktur data yang paling dasar dan penting adalah Linked List, khususnya Single Linked List (SLL). Berbeda dengan array yang memiliki ukuran tetap dan memori yang bersebelahan, Single Linked List menyimpan data dalam bentuk node-node yang terhubung melalui pointer, sehingga memori dapat dialokasikan secara dinamis sesuai kebutuhan. Setiap node pada SLL terdiri dari dua bagian yaitu data (nilai yang disimpan) dan pointer (referensi ke node berikutnya). Kelebihan utama Single Linked List adalah kemudahan dalam melakukan operasi penyisipan (insert) dan penghapusan (delete) pada posisi mana pun tanpa perlu menggeser elemen seperti pada array. Namun, kelemahannya adalah akses ke elemen tertentu memerlukan waktu yang lebih lama karena harus ditelusuri dari awal (sequential access).

Pada praktikum kali ini, SLL dipelajari melalui berbagai operasi dasar seperti penambahan node di depan, di akhir, dan di posisi tertentu, pencarian data, penghapusan node, serta penelusuran (traversal) seluruh elemen.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami konsep dasar Single Linked List dan cara kerjanya dalam menyimpan data secara dinamis dengan menggunakan node-node yang saling terhubung. Mahasiswa diharapkan mampu mengimplementasikan operasi-operasi dasar pada SLL seperti penambahan node di depan (insertAtFront), penambahan node di akhir (insertAtEnd), penambahan node di posisi tertentu (insertPos), serta penghapusan node di depan (deletehead), di akhir (removeLastNode), dan di posisi tertentu (deleteNode).

Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan dalam melakukan penelusuran (traversal) untuk mencetak seluruh isi linked list serta pencarian data (search) dalam SLL. Melalui keempat kode program yang diberikan (NodeSLL_2511533020, TambahSLL_2511533020, PencarianSLL_2511533020, dan HapusSLL_2511533020), mahasiswa dapat mengamati bagaimana node-node dihubungkan menggunakan pointer next_3020, bagaimana mengelola perubahan head ketika node ditambahkan atau dihapus, serta

bagaimana menangani kondisi batas seperti list kosong atau posisi di luar jangkauan. Pada akhir praktikum, mahasiswa diharapkan mampu membedakan Single Linked List dengan struktur data lain seperti array maupun Double Linked List.

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah penguasaan terhadap struktur data Single Linked List yang menjadi fondasi untuk struktur data lanjutan seperti Stack, Queue, Double Linked List, Circular Linked List, dan Graph. Dengan memahami SLL, mahasiswa dapat membuat program yang membutuhkan penyimpanan data dinamis dengan frekuensi penyisipan dan penghapusan yang tinggi, seperti manajemen daftar tugas (to-do list), antrian pada sistem pemrosesan, atau aplikasi playlist lagu. Selain itu, praktikum ini memberikan manfaat langsung dalam hal pemahaman tentang manipulasi pointer, pengelolaan memori dinamis, serta cara menelusuri dan mencari data dalam struktur berantai.

Pada program TambahSLL_2511533020, mahasiswa belajar bagaimana menambahkan node di berbagai posisi dengan memanipulasi pointer next_3020. Pada program HapusSLL_2511533020, mahasiswa belajar cara menghapus node dari berbagai posisi serta menangani kondisi di mana node yang dihapus adalah head atau node terakhir. Program PencarianSLL_2511533020 melatih kemampuan penelusuran dan pencarian data secara sequential. Semua ini menjadi bekal penting untuk mata kuliah lanjutan seperti Algoritma dan Pemrograman, serta Struktur Data Lanjut.

BAB II
PEMBAHASAN

1.1 Pengertian Single Linked List

Single Linked List (SLL) atau senarai berantai tunggal adalah struktur data linear yang terdiri dari sekumpulan node, di mana setiap node memiliki dua komponen yaitu data (nilai yang disimpan) dan pointer (atau referensi) yang menunjuk ke node berikutnya dalam urutan. Node terakhir dalam SLL akan memiliki pointer yang menunjuk ke null sebagai tanda akhir dari list.

Akses ke elemen dalam SLL hanya dapat dilakukan secara sequential, yaitu dimulai dari node pertama (disebut head) kemudian mengikuti pointer sampai node yang dituju. Kelebihan SLL adalah ukurannya yang dinamis (dapat bertambah atau berkurang sesuai kebutuhan) serta efisiensi dalam operasi penyisipan dan penghapusan karena tidak perlu menggeser elemen seperti pada array. Kekurangannya adalah akses acak (random access) tidak dimungkinkan, sehingga untuk mengakses elemen ke-n harus melalui node pertama hingga node ke-n (kompleksitas O(n)).

Selain itu, SLL memerlukan memori tambahan untuk menyimpan pointer pada setiap node. Operasi dasar pada SLL meliputi penambahan node (insert), penghapusan node (delete), penelusuran (traversal), dan pencarian (search). Dalam praktikum ini, semua operasi dasar tersebut diimplementasikan dalam bahasa Java.

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

2.2.1 NodeSLL_2511533020

```java
package pekan5_2511533020;

public class NodeSLL_2511533020 {
    // Node Bagian Data
    int data_3020;
    // Pointer ke node berikutnya
    NodeSLL_2511533020 next_3020;
    //Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533020 (int data_3020)
    {
        this.data_3020 = data_3020;
        this.next_3020 = null;
    }
}
```

(Gambar 2.2.1)

Kelas NodeSLL_2511533020 merupakan kelas dasar (blueprint) untuk membuat node-node dalam Single Linked List. Kelas ini memiliki dua atribut: data_3020 bertipe int untuk menyimpan nilai/nomor dari node, dan next_3020 bertipe NodeSLL_2511533020 yang berfungsi sebagai pointer (referensi) ke node berikutnya dalam list. Atribut next_3020 diinisialisasi dengan nilai null yang berarti bahwa node baru yang dibuat belum terhubung ke node manapun.

Konstruktor NodeSLL_2511533020(int data_3020) menerima parameter nilai yang akan disimpan, lalu mengisi atribut data_3020 dengan nilai tersebut dan mengatur next_3020 menjadi null. Kelas ini menjadi fondasi bagi semua operasi SLL lainnya, karena setiap node dalam list adalah objek dari kelas ini. Semua atribut menggunakan akhiran 3020 sesuai aturan 4 digit terakhir NIM

2.2.2 SLL_2511533020

```java
package pekan5_2511533020;

public class TambahSLL_2511533020 {
    public static NodeSLL_2511533020 insertAtFront (NodeSLL_2511533020 head_3020, int value_3020) {
        NodeSLL_2511533020 new_node = new NodeSLL_2511533020(value_3020);
        new_node.next_3020 = head_3020;
        return new_node;

    }
    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533020 insertAtEnd(NodeSLL_2511533020 head_3020, int value_3020) {
            //buat sebuah node dengan sebuah nilai
        NodeSLL_2511533020 newNode = new NodeSLL_2511533020(value_3020);
        // jika list kosong maka node jadi head
        if (head_3020 == null ) {
            return newNode;

        }
        //Simpan Head ke variabel sementara
        NodeSLL_2511533020 last = head_3020;
        //telusuri ke node akhir
        while (last.next_3020 != null) {
            last = last.next_3020;

        }
        //ubah pointer
        last.next_3020 = newNode;
        return head_3020;

    }
        static NodeSLL_2511533020 GetNode(int data_3020) {
            return new NodeSLL_2511533020  (data_3020);
            }

        static NodeSLL_2511533020 insertPos(NodeSLL_2511533020 headNode, int position_3020, int value_3020) {
            NodeSLL_2511533020 head_3020 = headNode;
            if (position_3020 < 1)
                System.out.print("Invalid Position");
```

```java
                    head_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(6);
                    //cetak list asli
                    System.out.print("Senarai berantai awal:");
                    printList(head_3020);
                    //tambahkan node baru di depan
                    System.out.print("tambah 1 simpul di depan:");
                    int data_3020 = 1;
                    head_3020 = insertAtFront(head_3020, data_3020);
                    //cetak update list
                    printList(head_3020);
                    // tambahkan node baru di belakang
                    System.out.print("tambah 1 simpul di belakang:");
                    int data2_3020 = 7;
                    head_3020 = insertAtEnd(head_3020, data2_3020);
                    //cetak update list
                    printList(head_3020);
                    System.out.print("tambah 1 simpul ke data 4:");
                    int data3_3020 = 4;
                    int pos_3020 = 4;
                    head_3020 = insertPos(head_3020, pos_3020, data3_3020);
                    // cetak update list
                    printList(head_3020);

                }

            }
```

```java
77                      head_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(6);
78                      //cetak list asli
79                      System.out.print("Senarai berantai awal:");
80                      printList(head_3020);
81                      //tambahkan node baru di depan
82                      System.out.print("tambah 1 simpul di depan:");
83                      int data_3020 = 1;
84                      head_3020 = insertAtFront(head_3020, data_3020);
85                      //cetak update list
86                      printList(head_3020);
87                      // tambahkan node baru di belakang
88                      System.out.print("tambah 1 simpul di belakang:");
89                      int data2_3020 = 7;
90                      head_3020 = insertAtEnd(head_3020, data2_3020);
91                      //cetak update list
92                      printList(head_3020);
93                      System.out.print("tambah 1 simpul ke data 4:");
94                      int data3_3020 = 4;
95                      int pos_3020 = 4;
96                      head_3020 = insertPos(head_3020, pos_3020, data3_3020);
97                      // cetak update list
98                      printList(head_3020);
99
100                 }
101
102             }
```

(Gambar 2.2.2)

Kelas TambahSLL_2511533020 berisi berbagai method untuk menambahkan node ke dalam Single Linked List. Method insertAtFront digunakan untuk menambahkan node baru di posisi paling depan (sebelum head). Method ini membuat node baru, mengatur next_3020 node baru agar menunjuk ke head saat ini, lalu mengembalikan node baru sebagai head baru. Method insertAtEnd digunakan untuk menambahkan node baru di posisi paling belakang. Method ini bekerja dengan cara: jika list kosong (head == null), maka node baru menjadi head; jika tidak, program menelusuri list sampai menemukan node terakhir (yang next_3020-nya null), lalu mengatur next_3020 dari node terakhir tersebut agar menunjuk ke node baru.

Method insertPos digunakan untuk menambahkan node pada posisi tertentu (misalnya posisi 4). Method ini memeriksa apakah posisi valid, kemudian jika posisi adalah 1 (depan), ia memanggil mekanisme insert di depan; jika tidak, program menelusuri list sampai posisi yang dituju, lalu menyisipkan node baru dengan mengubah pointer node sebelumnya agar menunjuk ke node baru, dan node baru menunjuk ke node setelahnya.

Method printList digunakan untuk mencetak seluruh isi linked list dengan format data-->data-->data. Di method main, dibuat linked list awal berisi 2,3,5,6. Kemudian dilakukan penambahan node 1 di depan, node 7 di belakang, dan node 4 di posisi 4. Hasil akhir dari program ini adalah list 1-->2-->3-->4-->5-->6-->7.

```
Console
<terminated> TambahSLL_2511533020 [Java Application] C:\Users\denni\.p2\
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan:1-->2-->3-->5-->6
tambah 1 simpul di belakang:1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4:1-->2-->3-->4-->5-->6-->7
```

2.2.3 PencarianSLL_2511533020

```java
package pekan5_2511533020;

public class PencarianSLL_251153020 {
    static boolean searchKey_3020(NodeSLL_2511533020 head_3020, int key_3020) {
        NodeSLL_2511533020 curr_3020 = head_3020;
        while (curr_3020 != null) {
            if (curr_3020.data_3020 == key_3020)
                return true;
        curr_3020 = curr_3020.next_3020;    }
        return false; }
    public static void traversal (NodeSLL_2511533020 head_3020) {
        //mulai dari head
        NodeSLL_2511533020 curr_3020 = head_3020;
        //telusuri sampai pointer null
        while (curr_3020 != null) {
            System.out.print(" " + curr_3020.data_3020);
            curr_3020 = curr_3020.next_3020; }
        System.out.println(); }
        public static void main(String[] args) {
            NodeSLL_2511533020 head_3020 = new NodeSLL_2511533020(14);
            head_3020.next_3020 = new NodeSLL_2511533020(21);
            head_3020.next_3020.next_3020 = new NodeSLL_2511533020(13);
            head_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(30);
            head_3020.next_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(10);
            System.out.print("Penelusuran SLL : ");
            traversal(head_3020);
            //data yang akan dicari
            int key_3020 = 30;
            System.out.print("cari data " +key_3020+ " = ");
            if (searchKey_3020(head_3020, key_3020))
                System.out.println("ketemu");
            else
                System.out.println("tidak ada");


    }

}
```

(Gambar 2.2.3)

Kelas PencarianSLL_2511533020 berisi method untuk melakukan pencarian data (searching) dalam Single Linked List serta penelusuran (traversal) seluruh elemen. Method searchKey_3020 menerima parameter head node dan nilai yang dicari (key). Method ini bekerja dengan cara menelusuri list dari head hingga akhir menggunakan bantuan variabel curr_3020 (current). Selama curr_3020 tidak null, program membandingkan curr_3020.data_3020 dengan key.

Jika ditemukan kesamaan, method mengembalikan true. Jika sampai akhir list tidak ditemukan, method mengembalikan false. Method traversal digunakan untuk menelusuri dan mencetak seluruh data dalam linked list. Method ini bekerja dengan cara yang sama: mulai dari head, selama node tidak null, cetak nilai node, lalu pindah ke node berikutnya. Di method main, dibuat linked list berisi data 14,21,13,30,10. Kemudian program melakukan penelusuran (traversal) untuk mencetak semua data, lalu melakukan pencarian nilai 30. Karena 30 ditemukan dalam list, program mencetak "ketemu". Program ini menunjukkan bahwa pencarian dalam SLL bersifat sequential dan kompleksitasnya O(n).

```
<terminated> PencarianSLL_251153020 [Java Application]
Penelusuran SLL :  14 21 13 30 10
cari data 30 = ketemu
```

2.2.4 HapusSLL_2511533020

```java
package pekan5_2511533020;


public class HapusSLL_2511533020 {

    public static NodeSLL_2511533020 deletehead_3020(NodeSLL_2511533020 head_3020) {
        if (head_3020 == null) return null;
        head_3020 = head_3020.next_3020;
        return head_3020;
    }

    public static NodeSLL_2511533020 removeLastNode_3020(NodeSLL_2511533020 head_3020) {
        if (head_3020 == null) return null;
        if (head_3020.next_3020 == null) return null;
        NodeSLL_2511533020 secondLast_3020 = head_3020;
        while (secondLast_3020.next_3020.next_3020 != null) {
            secondLast_3020 = secondLast_3020.next_3020;
        }
        secondLast_3020.next_3020 = null;
        return head_3020;
    }

    public static NodeSLL_2511533020 deleteNode_3020(NodeSLL_2511533020 head_3020, int position_3020) {
        if (head_3020 == null) return head_3020;
        if (position_3020 == 1) {
            return head_3020.next_3020;
        }
        NodeSLL_2511533020 temp_3020 = head_3020;
        NodeSLL_2511533020 prev_3020 = null;
        for (int i_3020 = 1; temp_3020 != null && i_3020 < position_3020; i_3020++) {
            prev_3020 = temp_3020;
            temp_3020 = temp_3020.next_3020;
        }
        if (temp_3020 != null) {
            prev_3020.next_3020 = temp_3020.next_3020;
        } else {
            System.out.println("Data tidak ada");
        }
```

```java
        return head_3020;
    }

    public static void printList_3020(NodeSLL_2511533020 head_3020) {
        if (head_3020 == null) {
            System.out.println("List kosong");
            return;
        }
        NodeSLL_2511533020 curr_3020 = head_3020;
        while (curr_3020.next_3020 != null) {
            System.out.print(curr_3020.data_3020 + "-->");
            curr_3020 = curr_3020.next_3020;
        }
        System.out.print(curr_3020.data_3020);
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511533020 head_3020 = new NodeSLL_2511533020(1);
        head_3020.next_3020 = new NodeSLL_2511533020(2);
        head_3020.next_3020.next_3020 = new NodeSLL_2511533020(3);
        head_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(4);
        head_3020.next_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(5);
        head_3020.next_3020.next_3020.next_3020.next_3020.next_3020 = new NodeSLL_2511533020(6);

        System.out.println("List awal: ");
        printList_3020(head_3020);

        head_3020 = deletehead_3020(head_3020);
        System.out.println("List setelah head dihapus: ");
        printList_3020(head_3020);

        head_3020 = removeLastNode_3020(head_3020);
        System.out.println("List setelah simpul terakhir dihapus: ");
        printList_3020(head_3020);

        int position_3020 = 2;
        head_3020 = deleteNode_3020(head_3020, position_3020);
```

```java
        head_3020 = deleteNode_3020(head_3020, position_3020);
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_3020(head_3020);
    }
}
```

(Gambar 2.2.4)

Kelas HapusSLL_2511533020 berisi berbagai method untuk menghapus node dari Single Linked List. Method deletehead_3020 digunakan untuk menghapus node paling depan (head). Method ini bekerja dengan cara: jika head tidak null, maka head digeser ke node berikutnya (head = head.next_3020), lalu head baru dikembalikan. Jika head null, method mengembalikan null. Method removeLastNode_3020 digunakan untuk menghapus node

paling belakang. Method ini bekerja dengan cara: jika list kosong atau hanya memiliki satu node, return null; jika tidak, program menelusuri list sampai menemukan node kedua dari terakhir (secondLast), lalu mengatur next_3020 dari node tersebut menjadi null sehingga node terakhir tidak lagi terhubung.

Method deleteNode_3020 digunakan untuk menghapus node pada posisi tertentu (parameter position). Method ini bekerja dengan cara: jika posisi 1, maka hapus head dengan mengembalikan head.next_3020. Jika posisi lebih dari 1, program menelusuri list sampai posisi yang dituju, dengan menyimpan node sebelumnya (prev). Setelah node yang akan dihapus ditemukan, maka prev.next_3020 diatur agar menunjuk ke node setelah node yang dihapus (temp_3020.next_3020). Method printList_3020 mencetak seluruh isi linked list. Di method main, dibuat linked list berisi 1,2,3,4,5,6.

Kemudian dilakukan penghapusan head (1), penghapusan node terakhir (6), dan penghapusan node pada posisi 2 (awalnya 2 menjadi 3). Hasil akhir setelah semua penghapusan adalah list 3-->4-->5. Program ini menunjukkan berbagai cara menghapus node dalam SLL serta penanganan kondisi batas seperti list kosong atau posisi tidak valid.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Single Linked List yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data linear dinamis yang terdiri dari node-node saling terhubung melalui pointer next. Keempat program yang diberikan berhasil menunjukkan operasi-operasi dasar SLL yaitu penambahan node di depan (insertAtFront), di belakang (insertAtEnd), dan di posisi tertentu (insertPos), penghapusan node di depan (deletehead), di belakang (removeLastNode), dan di posisi tertentu (deleteNode), serta penelusuran (traversal) dan pencarian data (searchKey).

Pemahaman tentang manipulasi pointer sangat penting dalam SLL, terutama saat menyisipkan atau menghapus node di tengah list. Kekurangan utama SLL adalah akses yang bersifat sequential, sehingga pencarian dan akses ke elemen tertentu memerlukan waktu O(n). Meskipun demikian, SLL tetap menjadi struktur data fundamental yang wajib dikuasai sebelum mempelajari struktur data yang lebih kompleks seperti Double Linked List, Stack, Queue, dan Graph.

3.2 Saran

Untuk pengembangan lebih lanjut, disarankan agar program TambahSLL_2511533020 dan HapusSLL_2511533020 dilengkapi dengan penanganan kondisi batas yang lebih baik, misalnya menambahkan pesan error ketika posisi yang dimasukkan melebihi jumlah node. Pada method insertPos di TambahSLL_2511533020, terdapat sedikit kekurangan dalam logika perulangan dan penanganan posisi di luar jangkauan; sebaiknya diperbaiki agar lebih robust. Selain itu, praktikum berikutnya dapat melanjutkan dengan implementasi Double Linked List (DLL) yang memiliki pointer prev dan next sehingga memungkinkan penelusuran dua arah, atau implementasi Circular Linked List.

Semua program sudah memenuhi aturan penamaan dengan akhiran NIM 3020 pada atribut dan variabel lokal yang relevan.

3.3 Penutup

Demikian laporan praktikum Struktur Data dengan judul Single Linked List ini disusun. Laporan ini mencakup penjelasan teoritis, implementasi kode, serta analisis dari empat

program yang telah dibuat, yaitu NodeSLL_2511533020, TambahSLL_2511533020, PencarianSLL_2511533020, dan HapusSLL_2511533020. Semoga laporan ini bermanfaat untuk memahami konsep Single Linked List dan penerapannya dalam bahasa pemrograman Java.

Penulis menyadari masih terdapat kekurangan, oleh karena itu kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatan

DAFTAR PUSTAKA

[1] Oracle, "LinkedList (Java Platform SE 17)," Oracle. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedList.html (diakses: 7 Mei 2026).

[2] W3Schools, "Java LinkedList," W3Schools. [Online]. Available: https://www.w3schools.com/java/java_linkedlist.asp (diakses: 7 Mei 2026).

[3] Wikipedia, "Linked list," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Linked_list (diakses: 7 Mei 2026).

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

TUGAS PRAKTIKUM

- **KELAS ADT:**

```java
package pekan5;

public class Pasien_2511533020 {
    // Atribut
    private String namaPasien_3020;
    private String keluhan_3020;
    private int nomorAntrian_3020;
    private Pasien_2511533020 next_3020;  // Pointer ke node berikutnya

    // Constructor
    public Pasien_2511533020(String namaPasien_3020, String keluhan_3020,
int nomorAntrian_3020) {
        this.namaPasien_3020 = namaPasien_3020;
        this.keluhan_3020 = keluhan_3020;
        this.nomorAntrian_3020 = nomorAntrian_3020;
        this.next_3020 = null;
    }

    // Selektor (Getter)
    public String getNamaPasien_3020() {
        return namaPasien_3020;
    }

    public String getKeluhan_3020() {
        return keluhan_3020;
    }

    public int getNomorAntrian_3020() {
        return nomorAntrian_3020;
    }

    public Pasien_2511533020 getNext_3020() {
        return next_3020;
    }

    // Mutator (Setter)
    public void setNamaPasien_3020(String namaPasien_3020) {
        this.namaPasien_3020 = namaPasien_3020;
    }

    public void setKeluhan_3020(String keluhan_3020) {
        this.keluhan_3020 = keluhan_3020;
    }

    public void setNomorAntrian_3020(int nomorAntrian_3020) {
        this.nomorAntrian_3020 = nomorAntrian_3020;
    }

    public void setNext_3020(Pasien_2511533020 next_3020) {
        this.next_3020 = next_3020;
    }
```

```java

    // Method toString untuk menampilkan data pasien
    @Override
    public String toString() {
        return "No. " + nomorAntrian_3020 + " | Nama: " + namaPasien_3020 + " | Keluhan: " + keluhan_3020;
    }
}
```

Kelas ini merepresentasikan seorang pasien dalam antrian. Atribut yang dimiliki: namaPasien_3020 (String), keluhan_3020 (String), nomorAntrian_3020 (int, diisi otomatis oleh sistem), dan next_3020 (pointer ke node pasien berikutnya). Semua atribut menggunakan akhiran 3020 sesuai aturan. Kelas ini dilengkapi dengan constructor, getter, setter, dan override toString() untuk memudahkan pencetakan data.

KELAS DRIVER

```java
package pekan5_2511533020;


public class HapusSLL_2511533020 {

    public static NodeSLL_2511533020 deletehead_3020(NodeSLL_2511533020 head_3020) {
        if (head_3020 == null) return null;
        head_3020 = head_3020.next_3020;
        return head_3020;
    }

    public static NodeSLL_2511533020 removeLastNode_3020(NodeSLL_2511533020 head_3020) {
        if (head_3020 == null) return null;
        if (head_3020.next_3020 == null) return null;
        NodeSLL_2511533020 secondLast_3020 = head_3020;
        while (secondLast_3020.next_3020.next_3020 != null) {
            secondLast_3020 = secondLast_3020.next_3020;
        }
        secondLast_3020.next_3020 = null;
        return head_3020;
    }

    public static NodeSLL_2511533020 deleteNode_3020(NodeSLL_2511533020 head_3020, int position_3020) {
        if (head_3020 == null) return head_3020;
        if (position_3020 == 1) {
            return head_3020.next_3020;
        }
        NodeSLL_2511533020 temp_3020 = head_3020;
        NodeSLL_2511533020 prev_3020 = null;
        for (int i_3020 = 1; temp_3020 != null && i_3020 < position_3020; i_3020++) {
            prev_3020 = temp_3020;
            temp_3020 = temp_3020.next_3020;
        }
        if (temp_3020 != null) {
            prev_3020.next_3020 = temp_3020.next_3020;
        } else {
            System.out.println("Data tidak ada");
        }
```

```java
    /**
     * 2. Panggil Pasien (Delete Head / Hapus node depan)
     * Menampilkan data pasien yang dipanggil, lalu head digeser ke node berikutnya
     */
    public void panggilPasien_3020() {
        if (head_3020 == null) {
            System.out.println("Antrian kosong! Tidak ada pasien yang dapat dipanggil.");
            return;
        }
        Pasien_2511533020 pasienDipanggil = head_3020;
        head_3020 = head_3020.getNext_3020();
        System.out.println("=== PASIEN DIPANGGIL ===");
        System.out.println(pasienDipanggil.toString());
        System.out.println("Silakan pasien menuju loket pelayanan.");
    }

    /**
     * 3. Tampilkan Antrian (Display / Traversal)
     * Menelusuri linked list dari head hingga null
     */
    public void tampilkanAntrian_3020() {
        if (head_3020 == null) {
            System.out.println("Antrian kosong! Belum ada pasien terdaftar.");
            return;
        }
        System.out.println("\n=== DAFTAR ANTRIAN PASIEN ===");
        Pasien_2511533020 current_3020 = head_3020;
        int posisi = 1;
        while (current_3020 != null) {
            System.out.println("[" + posisi + "] " + current_3020.toString());
            current_3020 = current_3020.getNext_3020();
            posisi++;
        }
    }

    /**
     * 4. Cari Pasien (Search by Name - Case Insensitive)
     * Mencari pasien berdasarkan nama (tidak case sensitive)
```

```java
     */
    public void cariPasien_3020(String namaCari) {
        if (head_3020 == null) {
            System.out.println("Antrian kosong! Tidak ada pasien untuk dicari.");
            return;
        }
        boolean ditemukan_3020 = false;
        Pasien_2511533020 current_3020 = head_3020;
        int posisi = 1;
        while (current_3020 != null) {
            // Case Insensitive: mengabaikan huruf besar/kecil
            if (current_3020.getNamaPasien_3020().equalsIgnoreCase(namaCari)) {
                System.out.println("Pasien ditemukan pada antrian ke-" + posisi);
                System.out.println("Data: " + current_3020.toString());
                ditemukan_3020 = true;
                break;
            }
            current_3020 = current_3020.getNext_3020();
            posisi++;
        }
        if (!ditemukan_3020) {
            System.out.println("Pasien dengan nama \"" + namaCari + "\" tidak ditemukan dalam antrian.");
        }
    }

    /**
     * 5. Cek Status Antrian
     * Menampilkan jumlah total pasien dan informasi pasien terdepan
     */
    public void cekStatusAntrian_3020() {
        if (head_3020 == null) {
            System.out.println("Status Antrian: KOSONG");
            System.out.println("Belum ada pasien yang mendaftar.");
            return;
        }
        // Hitung jumlah pasien
        int jumlah = 0;
        Pasien_2511533020 current_3020 = head_3020;
```

```java
        while (current_3020 != null) {
            jumlah++;
            current_3020 = current_3020.getNext_3020();
        }
        System.out.println("=== STATUS ANTRIAN ===");
        System.out.println("Jumlah pasien dalam antrian: " + jumlah);
        System.out.println("Pasien terdepan (selanjutnya dipanggil): " + head_3020.getNamaPasien_3020());
        System.out.println("Nomor antrian terakhir yang didaftarkan: " + counterAntrian_3020);
    }

    /**
     * Method main (menu interaktif)
     */
    public static void main(String[] args) {
        Scanner input_3020 = new Scanner(System.in);
        RumahSakit_2511533020 rs = new RumahSakit_2511533020();
        int pilihan_3020;

        System.out.println("=== Antrian Rumah Sakit NIM: 2511533020 ===\n");

        do {
            System.out.println("\n1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien   (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien      (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3020 = input_3020.nextInt();
            input_3020.nextLine(); // kosongkan buffer

            switch (pilihan_3020) {
                case 1:
                    System.out.print("Masukkan Nama Pasien : ");
                    String nama = input_3020.nextLine();
                    System.out.print("Masukkan Keluhan     : ");
                    String keluhan = input_3020.nextLine();
                    rs.daftarkanPasien_3020(nama, keluhan);
```

```java
                    break;
                case 2:
                    rs.panggilPasien_3020();
                    break;
                case 3:
                    rs.tampilkanAntrian_3020();
                    break;
                case 4:
                    System.out.print("Masukkan Nama Pasien yang dicari: ");
                    String namaCari = input_3020.nextLine();
                    rs.cariPasien_3020(namaCari);
                    break;
                case 5:
                    rs.cekStatusAntrian_3020();
                    break;
                case 6:
                    System.out.println("Terima kasih telah menggunakan program antrian rumah sakit.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid! Silakan coba lagi.");
            }
        } while (pilihan_3020 != 6);

        input_3020.close();
    }
}
```

OUTPUT

```
Console
RumahSakit_2511533020 [Java Application] C:\Users\denni\.p2\pc
=== Antrian Rumah Sakit NIM: 2511533020 ===


1. Daftarkan Pasien (Insert)
2. Panggil Pasien   (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien      (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan     : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
```